

Segment-Dependent Action Chunk Length via a Video Stability Predictor

Complete Implementation Document, v0.2

Aryan Goyal

July 2026 (status and results section added July 20)

Abstract

This document describes the full plan for a system that changes the executed action chunk length of an imitation-learned robot policy at run time. The decision signal is a small video model that watches the last 16 camera frames and predicts whether the robot is currently in an open-loop stable segment or an open-loop unstable segment. In stable segments the policy executes a long chunk of actions without feedback. In unstable segments the policy replans at every step. The ground-truth stability labels come from a simulator, using a standard perturb-and-replay procedure that measures the finite-time Lyapunov exponent. Only one small network head is trained. The policy and the video encoder are reused without modification. This document explains the models, the labeling procedure, the datasets, the training recipes, the runtime controller, the experiments, the baselines, the risks, and the compute budget.

Contents

1	The problem in plain terms	3
1.1	The theory behind the two constraints	3
1.2	Why existing adaptive-chunking methods do not solve this	4
2	The models and how they fit together	4
2.1	The policy model (reused, never retrained)	4
2.2	The video model (frozen encoder plus a small trained head)	5
2.3	The world model (label source, not a runtime component)	5
2.4	How the signal chooses the chunk length	5
2.5	The full pipeline in one line	6
3	Labeling open-loop and closed-loop stability	6
3.1	The quantity being measured	6
3.2	The open-loop label (the primary label)	7
3.3	The closed-loop label (secondary, diagnostic)	7
3.4	How the probe window length K is chosen	7
3.5	Which states get labeled (coverage)	8
3.6	Segmentation is an output, not an input	9
3.7	Cost	9

4	Datasets	9
4.1	Two distinct data roles	9
4.2	The specific datasets	9
5	Training	11
5.1	What is trained and what is reused	11
5.2	Loss function for the stability head	11
5.3	Training procedure and cost	11
6	Experiments	11
6.1	Stage 0: synthetic sanity check (days, no GPU)	11
6.2	Stage 1a: the key plot (oracle labels, no predictor)	12
6.3	Stage 1b: predictor quality	12
6.4	Stage 1c: the full closed loop	12
6.5	Stage 2: replication and ablations	12
7	Baselines	12
8	Risks and mitigations	13
9	Compute	13
10	Code and checkpoints	13
11	Status and results as of July 20, 2026	13
11.1	Stage 0: validated, including the falsifiability direction	14
11.2	Labeling: complete on all planned channels	14
11.3	Stage 1a: the effect exists, and it has a direction	14
11.4	Stage 1b: the head works, but not for the reason we assumed	14
11.5	Stage 1c: a tie on tasks where a tie is the ceiling	15
11.6	Stage 2: redefined and now running	15
11.7	Deviations and still-open items	15
12	Summary	16

1 The problem in plain terms

An imitation-learned policy such as Diffusion Policy or ACT predicts a chunk of future actions at once. The system designer must choose how many of those predicted actions to actually execute before asking the policy again. Call this number k , the executed chunk length. If $k = 1$ the policy replans at every step, which is fully closed loop. If k is large the robot runs open loop for k steps between replans.

The common belief in the literature is that small k is always safe and large k is only a matter of efficiency. Recent theory shows this belief is wrong. The correct choice of k depends on a property of the environment, not of the policy:

- In **open-loop stable** segments (free-space reaching and transport, where the low-level controller pulls small errors back to the reference), the chunk must be *longer* than a threshold k^* . Short chunks are actively harmful here, because every replan injects the policy’s estimation error into the system, and per-step replanning injects it at every step.
- In **open-loop unstable** segments (contact events such as grasping and insertion, where small errors grow), long open-loop execution lets errors compound without correction. The chunk must be *short*, and per-step feedback is the right mode.

A long-horizon task contains both kinds of segments. Therefore no single constant k can be correct for the whole task. The hypothesis of this project is that k should be switched per segment, and that the switching signal should measure the stability of the environment rather than the confidence of the policy.

1.1 The theory behind the two constraints

The relevant results are from Simchowitz et al. (arXiv:2503.09722) and Zhang et al. (arXiv:2507.09061). Let $\mathbf{J}_{\text{TRAJ}}$ be the trajectory error of the learned policy at execution time and $\mathbf{J}_{\text{DEMO}}$ its regression error on the demonstration distribution. Exponential error compounding means

$$\mathbf{J}_{\text{TRAJ}} \gtrsim C^T \mathbf{J}_{\text{DEMO}}, \quad C > 1, \quad (1)$$

where T is the horizon. The stability notion is exponential incremental input-to-state stability (EISS): a system is (C_{ISS}, ρ) -EISS with $\rho \in (0, 1)$ if perturbations between two trajectories contract geometrically over time.

The theory splits into two cases:

1. **Stable dynamics, closed-loop execution still fails.** Even when the dynamics contract, a smooth Markovian policy that replans every step suffers exponential compounding. The feedback channel is the mechanism: the policy carries estimation error, and closing the loop injects that error at every step.
2. **Chunking fixes the stable case, if the chunk is long enough.** Executing k actions open loop means error is injected once per chunk instead of once per step, and the contraction absorbs it between injections. This works only above a threshold:

$$k > k^* = \log(1/\rho)^{-1} \cdot \log(\text{poly}(L_\pi, C_{\text{ISS}})), \quad (2)$$

where L_π is the policy Lipschitz constant. Above k^* the trajectory error is horizon-free, $\mathbf{J}_{\text{TRAJ}} \leq O_\star(1) \mathbf{J}_{\text{DEMO}}$. Below k^* the exponential lower bound still applies.

3. **Unstable dynamics cannot be fixed at execution time.** For open-loop unstable dynamics, exponential compounding is unavoidable for any policy trained only on expert data. The admissible fix is on the data side: injecting noise into the demonstrations (Theorem 2 of Zhang et al.) reduces the error growth from exponential to polynomial in T . In these segments short chunks and per-step feedback are the best execution-side option, and chunking is known to degrade performance empirically.

Applied segment by segment, the two constraints have opposite signs: contracting segment i requires $k \geq k_i^*$, while expansive segment j requires $k \lesssim 1/\lambda_j$ where λ_j is the local expansion rate. A constant k generically violates one of them.

1.2 Why existing adaptive-chunking methods do not solve this

Existing adaptive methods (AAC, DVAC, HiPolicy, BID, AQC, competency-adaptive execution) all condition on policy-side signals: action entropy, denoising variance, resampling coherence, or learned value estimates. These signals can only ever license commitment. They say that committing is permissible when the policy is confident. They cannot say that committing is *required*. The harm of not committing in a stable segment lives in the feedback channel, which is a property of the closed-loop transition map and is absent from the policy’s output distribution. A policy can be confident yet wrong in a contact segment, and uncertain in a benign free-space segment. Our signal measures the environment instead, so it can produce the minimum-length constraint that policy-side signals cannot produce even in principle. The disagreement cases between the two kinds of signal are the natural experiment, and we test them explicitly (Section 6).

2 The models and how they fit together

The system has three model components with strictly separated roles. Only one of them is trained in this project.

2.1 The policy model (reused, never retrained)

The policy is a standard chunking imitation policy:

- **Stage 1:** Diffusion Policy, from the public real-stanford repository, with the released robomimic image configurations. We use released checkpoints where available, or retrain from the released config, which takes hours on one GPU.
- **Stage 2:** ACT on the ALOHA simulated tasks, to show the result is not specific to one policy class.

The key design decision is that the executed chunk length k is an inference-time knob. The policy always predicts its full action chunk; we only vary how many predicted actions are executed before replanning. Because of this, the policy is never retrained, its loss is never modified, and every comparison in the project runs the exact same policy weights. Any performance difference between chunking strategies is therefore attributable to the execution rule alone.

2.2 The video model (frozen encoder plus a small trained head)

The trained component is a stability predictor. Its only job is: given the recent video, say whether the current segment is open-loop stable or unstable.

- **Input:** the last 16 camera frames, plus proprioception (joint readings).
- **Backbone:** V-JEPA 2 ViT-L, completely frozen (public checkpoints, `facebook/vjepa2-*`). The reason for a video backbone rather than an image backbone is that stability is a dynamics property. Contact, incipient slip, and constrained motion are motion cues. They are visible in a clip and not in a single frame. V-JEPA 2 is pretrained by masked latent video prediction, which is exactly a motion-predictive objective, so its features are suited to this readout.
- **Head:** attentive pooling followed by a 2-layer MLP, which is V-JEPA’s standard attentive-probe protocol, with reused code. Proprioception passes through a small MLP and is concatenated after pooling. This head is the only trained component in the entire project.
- **Outputs:** the probability that the current segment is open-loop unstable, $p(\text{OL-unstable})$; an auxiliary regression of the open-loop exponent $\hat{\lambda}_{\text{ol}}$; and optionally the closed-loop pair $p(\text{CL-unstable})$ and $\hat{\lambda}_{\text{cl}}$.
- **Ablations:** a single-frame DINOv2 backbone with the same head, to prove that the temporal information matters; and LoRA on the last encoder blocks in case the frozen probe underfits.

2.3 The world model (label source, not a runtime component)

The word “world model” plays two roles at two different times, and it is important to keep them separate:

- **Now, in simulation:** the “world model” is the MuJoCo simulator itself, used as a ground-truth oracle. It is not learned and it involves no gradients. We use `get_state` and `set_state` to reset the simulator to any recorded moment, perturb the first action, replay forward, and measure how fast trajectories diverge. This procedure generates the stability labels (Section 3).
- **Later, on a real robot:** there is no simulator to reset and replay, so a learned action-conditioned world model (V-JEPA 2-AC or DINO-WM) replaces the simulator as the label source. The world model imagines paired rollouts in its latent space: same actions, perturbed first action, and the label is the slope of the log latent divergence between the imagined rollouts. The labeling logic, the predictor architecture, and the losses are unchanged. Only the oracle that produces the labels is swapped. Hardware perturb-and-replay on a subsample of states is the backup option.

The world model is never in the control loop. At run time the only extra computation is the frozen encoder and the small head.

2.4 How the signal chooses the chunk length

At each replanning point the controller runs the encoder and head on the last 16 frames, reads out the predicted regime, and sets the executed chunk length:

Predicted regime	Executed chunk length	Meaning
OL-stable	$k = k_{\text{long}}$	run open loop; standard chunk (Diffusion Policy $T_a = 8$ to 16, ACT 100)
OL-unstable and CL-stable	$k = 1$	per-step feedback; Markovian mode
Neither (both unstable)	$k = 1$ plus a flag	no execution-side fix exists here; the remedy is data-side noise injection

Details of the rule:

- **Hysteresis:** the controller requires two consecutive agreeing predictions before it switches regime, to avoid chattering at segment boundaries.
- **MVP:** the minimum viable version collapses to the first two rows, a binary switch on the open-loop head, since the third case is rare in these benchmarks. Both heads are trained from the start, so the full three-case rule costs nothing extra.
- **No derived k^* :** we do not compute the theoretical threshold. The empirical k^* per segment comes from the Stage 1 sweep, defined as the smallest k at which success plateaus in stable segments.
- **Continuous rule as ablation:** the auxiliary $\hat{\lambda}$ regression gives a continuous rule $k \approx c/|\hat{\lambda}|$ for free. This is an ablation, not the main method.

2.5 The full pipeline in one line

Simulator oracle $\rightarrow$ per-timestep stability labels $\rightarrow$ supervised training of (frozen video encoder + small head) $\rightarrow$ at run time the predictor watches recent frames $\rightarrow$ binary switch of the executed chunk length.

3 Labeling open-loop and closed-loop stability

This section describes exactly how the ground-truth labels are produced. There is no learning anywhere in this procedure. It is a script, not a model.

3.1 The quantity being measured

The ground-truth quantity is the finite-time Lyapunov exponent (FTLE), the textbook estimator of local exponential divergence (Benettin et al. 1980; used for reinforcement learning policy robustness in arXiv:2410.10674). The question the label answers is: if a small error of realistic size enters the system right now, does it grow over the next segment or does it get absorbed?

The perturbation size is chosen to make this question exact. We set σ_u equal to the trained policy’s validation action RMSE, so the probe injects exactly the size of error the policy will actually make. With this choice the label directly measures the error-compounding gain from the theory: whether an injected error of realistic size is amplified or absorbed over the segment horizon.

The theory papers themselves (arXiv:2503.09722, arXiv:2507.09061) provide no empirical classification procedure. They assume EISS and build synthetic systems that are stable by construction. The per-segment labeling protocol below is ours, but every ingredient is standard and citable: the FTLE, contraction analysis (Lohmiller and Slotine 1998), and the imitation-learning community’s open-loop replay test.

3.2 The open-loop label (the primary label)

The open-loop label is a property of the plant: the robot, its low-level controller, and the environment together. For each demo timestep t , subsampled every 2 to 5 steps, we run the following in MuJoCo:

1. Reset the simulator to the recorded state at time t .
2. **Nominal branch:** replay the recorded actions $a_{t:t+K}$ exactly. The final recipe uses $K = 24$; how K is chosen is explained in Section 3.4.
3. **Perturbed branches:** run $N = 8$ additional branches, identical to the nominal branch except that Gaussian noise with standard deviation σ_u is added to the *first action only*. All later actions are replayed unchanged. The robot does not react in any branch; the actions are fixed.
4. Measure $d(\tau)$, the state-space distance between the perturbed branches and the nominal branch at each step τ , using end-effector pose plus object pose. Fit

$$\lambda_{\text{ol}}(t) = \text{OLS slope of } \log d(\tau) \text{ versus } \tau. \quad (3)$$

5. Threshold with a deadband:

$$y_{\text{ol}}(t) = \begin{cases} \text{unstable} & \lambda_{\text{ol}}(t) > +\delta \\ \text{stable} & \lambda_{\text{ol}}(t) < -\delta \\ \text{inherit from neighbor} & \text{otherwise.} \end{cases} \quad (4)$$

Then apply a median filter over about 3 consecutive label stamps. The result is contiguous stable and unstable segments. Simulator contact flags are logged as an independent sanity channel, never as a label.

The intuition is simple. If the nudged replays drift away from the clean replay, errors grow on their own and the segment is unstable. If the nudged replays converge back to the clean replay, the plant absorbs errors and the segment is stable.

3.3 The closed-loop label (secondary, diagnostic)

The closed-loop label uses the same perturbed pairs, with one change: in the perturbed branch the trained policy acts from its own observations and replans at every step, instead of replaying fixed actions. This yields $\lambda_{\text{cl}}(t)$ and $y_{\text{cl}}(t)$ by the same fitting and thresholding procedure.

The comparison of λ_{cl} against λ_{ol} per segment verifies the mechanism from the theory: in contracting segments, feedback injects estimation error and should make things worse relative to open-loop replay; in expansive segments the ordering should flip. The closed-loop head also enables the third row of the runtime rule.

3.4 How the probe window length K is chosen

K is the number of steps we watch the error for after the nudge. It is not arbitrary: it is bracketed from below by two requirements and from above by two others, and then fixed empirically inside that bracket.

Lower bound 1: the fit must see past the controller transient. λ is the OLS slope of $\log d(\tau)$ against τ . The first 0.2 to 0.5 seconds after the nudge (4 to 10 steps at 20 Hz) are dominated by the low-level tracking controller settling, regardless of the regime. If K barely exceeds this transient, the fitted slope measures the transient rather than the plant regime. K must leave enough points beyond it for a stable line fit.

Lower bound 2: the window must cover the decision it certifies. The label’s runtime meaning is “it is safe to commit a chunk of actions open loop starting here.” A verdict measured over a window shorter than the chunk would not certify the commitment being made. So K must be at least the longest chunk the controller may execute: $K \geq k_{\text{long}} = T_p = 16$ for Diffusion Policy (0.8 seconds at 20 Hz).

Upper bound 1: locality. The label is a per-segment quantity. If K approaches the typical dwell time of a segment, the window straddles regime boundaries and the fitted λ blends a contracting stretch with an expansive one. The median filter cannot repair a window that is itself mixed. K must therefore stay well below segment length.

Upper bound 2: the fit must stay in the exponential regime. The estimator assumes $d(\tau) \approx \sigma_u e^{\lambda\tau}$, which holds only while the perturbation stays small relative to the workspace and no discrete event (a new contact, a constraint activation) redirects the divergence. Over long windows $d(\tau)$ saturates and $\log d$ flattens, biasing λ toward zero exactly at the unstable stamps we most need to detect. A larger σ_u reaches saturation sooner, so this bound couples K to σ_u : with σ_u set to the policy RMSE, K cannot be grown freely.

The empirical pick inside the bracket. The admissible range for our setup is roughly $16 \leq K \lesssim 30$. A six-variant probe study on 30 Square demos selected $K = 24$ (1.2 seconds) with position-only perturbation: 36.6% of fixture-contact stamps sit above the deadband versus 12.7% at $K = 16$, while free-space false positives stay at 1.2%. The Stage 0 synthetic system provides the correctness check: at these settings the labeler recovers the true λ of a known piecewise-linear system to four decimal places.

K is a per-embodiment constant, set by the controller settling time and the chunk horizon, not a per-task tune. It is re-derived once for ALOHA in stage 2 (different controller, different control rate), and the label hyperparameters N , K , σ_u , δ get a sensitivity ablation in stage 2.

3.5 Which states get labeled (coverage)

Labels are generated along two kinds of trajectories:

1. **Demonstration states:** every demo trajectory, subsampled as above. The replayed actions are the demo’s recorded actions.
2. **Policy rollout states, including failed rollouts:** trajectories produced by running the trained policy. Here the replayed actions in the nominal branch are the policy’s own executed actions.

The reason for including rollouts is distribution match. The predictor will run on the states the policy actually visits, which include states outside the clean demo tube, especially during recoveries and failures. Labeling only demos would train the predictor on a narrower distribution than it faces at run time.

3.6 Segmentation is an output, not an input

We never segment trajectories by hand or by semantic phase. All datasets are used as continuous, unannotated trajectories. The per-timestep FTLE labels, after thresholding and median filtering, *are* the segments. Stability regime is the only segment notion the hypothesis uses. Whether the stability segments happen to align with semantic phases such as reach, grasp, and insert is a qualitative result we report, not an assumption we rely on.

3.7 Cost

The labeler runs pure simulator rollouts with no gradients, and every labeled timestep is independent, so the work is embarrassingly parallel. The total is about 200 demos plus a similar number of policy rollouts, times one label every 2 to 5 steps, times $(N + 1) = 9$ branches of $K = 16$ steps each. This is hours on CPU workers.

4 Datasets

4.1 Two distinct data roles

The datasets serve two different purposes, and the requirements differ:

1. **Predictor training pool: all tasks, short and long.** The stability label is local. It depends only on the recent frames and the local regime, so episode length is irrelevant to the label’s meaning. Short tasks add cheap regime diversity, and they break a shortcut the predictor could otherwise learn, namely that late in an episode implies contact.
2. **Hypothesis-test tasks: long-horizon only.** The claim under test is that no constant k works when a task contains both regimes. Only tasks with both regimes can show this. Single-regime short tasks cannot dissociate the hypothesis, but they make clean controls: a mostly-stable task should favor long chunks or tie, and a contact-heavy task should favor $k = 1$ or tie.

4.2 The specific datasets

The datasets are organized by the role they play. The rule that governs the split: the predictor training pool feeds the predictor only, and the held-out evaluation tasks never enter training.

Predictor training pool (Panda arm with OSC control, one shared predictor).

- **robomimic Lift-ph, Can-ph, Square-ph, Tool-Hang-ph:** all four 200-demo proficient-human sets.
- **MimicGen D0–D1** generated sets across its task suite, including the short tasks (coffee, threading, stack) for contact-event diversity.
- **FurnitureSim one_leg** demonstrations.

Label-quality evaluation (calibration and generalization, not policy rollout).

- Calibration against sim ground-truth FTLE: **Square-ph** and **Tool-Hang-ph**.
- Mostly-stable control task: **Can-ph**.
- Task-transfer test: head trained on Square only, labels scored on Tool-Hang. This tests whether the video model learned stability itself or a task-specific visual cue.
- Embodiment-transfer tier (stage 2): **ALOHA** simulated insertion and transfer-cube, with a separate action head and labels recomputed from scratch. Temporal ensembling stays off by default because it reinstates per-step feedback; turning it on and off is an ablation.

Policy rollout, short horizon.

- **Square-ph** and **Tool-Hang-ph** (primary; reference Diffusion Policy numbers).
- **Can-ph** as the negative control: the adaptive schedule should show approximately zero gap over the best fixed chunk length here.

Policy rollout, long horizon.

- **MimicGen** `three_piece_assembly`, `nut_assembly`, `kitchen`, `coffee_preparation`: these span an alternation-count axis (how many stable-to-unstable transitions per episode) and produce the headline scaling curve.
- **FurnitureSim** `one_leg`: the contact-heavy showcase.
- **LIBERO-Long**, paired with **LIBERO-Pro** perturbation levels: a reviewer-legible benchmark with measured headroom.

Excluded, deliberately.

- robomimic **Transport**: bimanual, demoted to stage 2.
- **CALVIN**: excluded on the TTIC audit consistency findings.

Priority order. Same-embodiment (Panda/OSC) work comes first: the robomimic ph tasks (running), then MimicGen, then FurnitureSim. LIBERO and ALOHA follow after the same-embodiment results land.

Real robot (deferred). Same predictor architecture; labels come from world-model rollout divergence as described in Section 2, or from hardware perturb-and-replay on a subsample.

5 Training

5.1 What is trained and what is reused

Component	Source	Status
Chunking policy	Diffusion Policy (real-stanford), ACT	Reused; retrain from released config only if needed
Video encoder	V-JEPA 2 ViT-L, frozen	Reused, frozen
Stability head	attentive probe + MLP + proprio MLP	Built and trained (only one)
Label generator	MuJoCo perturb-and-replay script	Built (a script, not a model)
Benchmarks and demos	robomimic, ALOHA sim, FurnitureSim	Reused
Real-world label source	V-JEPA 2-AC or DINO-WM divergence	Deferred

5.2 Loss function for the stability head

$$L = \text{BCE}_w(p_{\text{ol}}, y_{\text{ol}}) + \alpha \text{Huber}(\hat{\lambda}_{\text{ol}}, \lambda_{\text{ol}}) + \beta (\text{the same pair of terms for CL}). \quad (5)$$

- **Class-weighted BCE** on the open-loop regime is the decision loss. Unstable timesteps are the minority class, so we weight the loss or balance the sampling.
- **Huber** on λ is auxiliary. It calibrates the head, and it provides the continuous- k rule ($k \approx c/|\hat{\lambda}|$) for free as a later ablation. We start with $\alpha = 0.5$.
- **Closed-loop terms** are trained from the start with $\beta = 0.5$. They enable the full three-case runtime rule and the mechanism analysis, at negligible cost.
- **Policy losses are untouched.** Diffusion Policy keeps its DDPM MSE loss and ACT keeps its L1 plus KL loss. The encoder stays frozen throughout.

5.3 Training procedure and cost

The encoder is frozen, so encoder features for every clip can be computed once and cached. Training the head on cached features takes minutes to hours. Policy training, when needed at all, is at most about 12 hours per task on a single A100 using the released configurations, and released checkpoints avoid even that. Label generation is CPU hours. Stage 0 runs on a laptop. Stages 0 and 1 together are estimated at 4 to 6 weeks wall-clock.

6 Experiments

The experiment ladder produces one decisive plot per stage.

6.1 Stage 0: synthetic sanity check (days, no GPU)

A synthetic piecewise-linear system with three segments of known contraction rates, taken from the caveat section of the hypothesis note. We sweep chunk length per segment. Prediction: the optimal chunk-length profile tracks the contraction profile, with a floor k_i^* in contracting segments and a ceiling near $1/\lambda_j$ in expansive segments, and success degrades as $k \rightarrow 1$ in stable segments. This validates the segment-wise application of the theory before any robot experiment.

6.2 Stage 1a: the key plot (oracle labels, no predictor)

A forced- k -per-segment sweep on Square and Tool-Hang using the oracle labels directly. The predictor is not in the loop yet, which isolates the hypothesis from predictor quality. We plot a heatmap of task success over the grid $(k_{\text{stable}}, k_{\text{unstable}})$.

Prediction: the best corner is (large, small), and the $k_{\text{stable}} = 1$ column is *bad*. The second half of this prediction is the sign prediction that no other method in the literature makes, because every policy-side signal treats $k = 1$ as universally safe.

6.3 Stage 1b: predictor quality

AUROC and calibration of $p(\text{OL-unstable})$ against held-out oracle labels; alignment with simulator contact flags; cross-task generalization (train on Square, evaluate on Tool-Hang); and the video-versus-single-frame ablation (V-JEPA 2 against DINOv2 with the same head).

6.4 Stage 1c: the full closed loop

The predictor-driven switch against every fixed k and against the policy-side baselines of Section 7, all running the same policy weights. This stage includes the two dissociation cases where environment-side and policy-side signals disagree:

1. **Contact with clean unimodal visuals.** The policy is confident, but the environment is unstable. Entropy and variance baselines over-commit; our predictor shortens the chunk.
2. **Free space with induced bimodal demonstrations.** The policy is uncertain, but the environment is stable. The baselines shorten the chunk, and by the theory this per-step replanning is what fails; our predictor commits to a long chunk.

6.5 Stage 2: replication and ablations

ACT on ALOHA; a long-horizon task (Transport or FurnitureSim); and the ablations: the continuous- k rule, temporal ensembling on and off, noise injection into unstable-segment demonstrations per Theorem 2, and the label hyperparameters N , K , σ_u , δ .

7 Baselines

All baselines are inference-time rules on the same trained Diffusion Policy, so every comparison holds the policy fixed:

- **Fixed $k \in \{1, 2, 4, 8, 16\}$.** The full constant-chunk sweep.
- **AAC-style.** Execute the low-entropy prefix of each chunk. The Diffusion Policy proxy for entropy is the variance across resampled chunks.
- **DVAC-style.** Truncate the chunk when denoising variance rises.
- **BID-style.** Per-step resampling.

8 Risks and mitigations

Risk	Mitigation
Label noise near segment boundaries	Deadband δ , median filter, subsample stride.
Perturbation scale wrong: too small hits the noise floor, too large leaves the linear regime	σ_u set to the policy validation RMSE, plus a sensitivity ablation.
Class imbalance: unstable is the minority class	Weighted BCE or balanced sampling.
Predictor learns a task-position shortcut (late in episode implies contact) instead of dynamics	Cross-task generalization test; short tasks in the training pool; shuffle-clip control.
ACT temporal ensembling reinstates per-step feedback and confounds the comparison	Off by default, ablated explicitly.
Simulator labels may not transfer to the real world	World-model-divergence labels replace the oracle later; same head, same losses.

9 Compute

A single A100 or less is sufficient. Diffusion Policy training is at most about 12 hours per task, or zero when released checkpoints exist. Label generation is CPU hours and embarrassingly parallel. Head training is minutes to hours on cached frozen features. Stage 0 is a laptop experiment. Stages 0 and 1 are estimated at 4 to 6 weeks wall-clock.

10 Code and checkpoints

- Diffusion Policy: github.com/real-stanford/diffusion_policy (robomimic image configs).
- robomimic datasets: robomimic.github.io (square, tool_hang, can; ph variants).
- V-JEPA 2, attentive-probe code, and the AC variant for later: github.com/facebookresearch/vjepa2.
- DINOv2 for the single-frame ablation: [facebook/dinov2-base](https://facebook.com/dinov2-base).
- ACT and ALOHA sim: github.com/tonyzhaozh/act.
- π_0 for the real-robot stage: github.com/Physical-Intelligence/openpi.

11 Status and results as of July 20, 2026

This plan was written before execution. Everything through Stage 1 has now run, and Stage 2 is running. This section records, stage by stage, what was executed, what was found, and

where execution deviated from the plan above. It is a summary layer; the full evidence lives in four companion documents: the labeling report (`label_report_latex`), the Stage 1a report (`stage1a_report_latex`), the predictor document (`predictor_doc_latex`), and the Stage 2 document (`stage2_doc_latex`).

11.1 Stage 0: validated, including the falsifiability direction

The synthetic piecewise-linear system behaved exactly as Section 6 predicted. Contracting segments showed a floor near $k^* \approx 5$ to 6 (chunks of 3 or less drove success to zero), expansive segments showed a ceiling near 4 to 5 (chunks of 8 or more drove success to zero), and with those parameters the best constant k reached 3 percent success while the adaptive profile reached 100 percent. The FTLE labeler recovered the true λ to four decimals, and the closed-loop λ flipped sign correctly in both regimes. A second run with milder parameters showed a constant k sufficing, which documents the falsifiable boundary: adaptive chunking wins exactly when the stable-segment floor exceeds the unstable-segment ceiling, and not otherwise.

11.2 Labeling: complete on all planned channels

The open-loop perturb-and-replay procedure of Section 3 ran to completion: about 155,000 labeled stamps across the four robomimic tasks (demonstrations plus policy rollouts) and LIBERO. The closed-loop channel ran on all four robomimic tasks (3,969 stamps) and confirmed the mechanism premise on every task: perturbations injected under closed-loop replanning keep growing, with positive closed-loop λ at 90 to 99 percent of stamps per task. One planning error is worth recording: the plan called labeling “CPU hours”; in practice it was GPU days, with the closed-loop tool_hang channel alone taking 54 hours.

11.3 Stage 1a: the effect exists, and it has a direction

The forced- k grid ran on square and tool_hang, 12 cells, three seeds, 150 episodes per cell, with simulator contact plus hysteresis as the oracle signal. Square is flat (0.773 to 0.873, nothing clears two sigma): the base policy evaluates at 0.90 and there is no headroom, which is itself the lesson that Stage 2 task selection acted on. Tool_hang produced the project’s first rollout-level effect: per-step replanning during contact pools to 0.60 to 0.67 against a 0.79 to 0.81 plateau for committed chunks, at three to five sigma in three independent cells. The plan’s sign prediction, that the $k=1$ column would be bad, held (fixed $k=1$ reaches 0.667 on tool_hang against 0.807 for the best fixed k). The surprise is where the damage concentrates: in contact, the segment the oracle calls unstable. That is the direction the closed-loop labeling channel predicted (closed-loop λ stays positive in contact, so per-step feedback does not stabilize these segments), and it contradicts the naive reading that unstable segments simply want $k=1$. The binary oracle switch itself never beat the best fixed k ; the explanation is that these rollouts are 65 to 88 percent contact, so a binary signal leaves the switch almost nothing to modulate. That finding strengthened, rather than weakened, the case for the continuous predicted λ .

11.4 Stage 1b: the head works, but not for the reason we assumed

The pooled head reaches AUROC 0.87 on confident held-out stamps (0.89 with LIBERO in the pool) with λ MAE 0.027. Two planned tests came back with answers the plan did not anticipate. Cross-task generalization fails outright (AUROC 0.56 training on one task and evaluating on

another), so pooled training is required, and the pooled result is what everything downstream uses. And the planned video-versus-single-frame ablation returned a tie: a single-frame DINOv2 head matches the 16-frame V-JEPA 2 head almost exactly (0.703/0.859 against 0.699/0.868), so the signal the head reads is scene configuration plus proprioception, not motion. The shuffle-clip control from the risks table became moot and was dropped: a single frame cannot exploit temporal order, yet it ties the clip. A head trained on the closed-loop labels (beyond the plan) reached only 0.627, consistent with closed-loop expansion being a property of the policy’s reaction rather than of the visible scene.

11.5 Stage 1c: a tie on tasks where a tie is the ceiling

The predictor-driven (16, 4) switch pooled over three seeds: `tool_hang` 0.800, a statistical tie with both the best fixed k (0.807) and the best oracle cell (0.807), reached while replanning adaptively (mean executed k near 11) and avoiding the per-step trap. Square pools to 0.767 inside the flat band, the expected null at ceiling. A live perturb-and-replay probe cell was run once as an in-the-loop oracle but degenerated to fixed $k=16$ because its threshold scale does not transfer from the labeling protocol; it is recorded and de-prioritized. The Stage 1 conclusion, stated plainly: on short contact-dominated tasks the best fixed k is near-optimal and matching it is the achievable outcome; the decisive test of adaptivity needs long tasks that alternate between stable transport and unstable contact. That conclusion redefined Stage 2.

11.6 Stage 2: redefined and now running

The Stage 2 of Section 6 (ACT on ALOHA, execution-rule ablations) is deferred, not dropped. In its place, Stage 2 is the long-horizon test: Diffusion Policy on four MimicGen tasks (`three_piece_assembly`, `nut_assembly`, `kitchen`, `coffee_preparation`, D0, 1,000 demonstrations each, mean episode lengths 337 to 689 steps), followed by the same fixed- k versus predictor-switch comparison. The first training runs were killed at 80 to 86 percent of their 2,000-epoch budget by a scratch-disk failure early on July 21; checkpoints saved every 50 epochs to persistent storage, so selection proceeds on the surviving grids (which end past the quality plateau) while fresh full-length reruns provide insurance in parallel. The experiment timeline moved by less than a day: checkpoint selection the evening of July 21, the switch-versus-fixed- k rollouts on July 22. The design, the simulator version conflict that shapes the infrastructure, the disk incident, and the zero-shot decision for the head are documented in the Stage 2 companion document.

11.7 Deviations and still-open items

- **Compute.** The plan’s estimates were wrong in both directions of kind: Diffusion Policy training is about five to six days per task at the 200k-step budget, not 12 hours, and labeling was GPU days, not CPU hours. The work ran on an 8-GPU machine, not a single A100.
- **DINOv2 variant.** The ablation used `dinov2-large`, not the `dinov2-base` listed in the code and checkpoints section.
- **Not yet run.** The policy-side baselines (AAC, DVAC, BID styles), the two dissociation cases of Stage 1c, the can negative control, the continuous- k rule, the temporal-ensembling and noise-injection ablations, and the label-hyperparameter sweeps. These remain valid follow-ups; none blocks Stage 2.

- **Parked.** FurnitureSim (manual download requirement); LIBERO-Long policy training is an open option for currently idle GPUs.

12 Summary

The project trains exactly one small network: a head on a frozen video encoder that predicts whether the current moment is open-loop stable or unstable. The labels come from a simulator procedure that nudges the first action by the size of error the policy actually makes and fits the slope of the log divergence. The labels themselves define the segments; nothing is hand-segmented. At run time the predictor switches the executed chunk length: long chunks in stable segments, per-step feedback in unstable ones. The policy is never modified, so every comparison isolates the execution rule. The central testable prediction, which no policy-confidence method can make, is that short chunks actively fail in stable segments. Stage 1a tested exactly that with oracle labels before the predictor was ever in the loop; Section 11 records the outcome, including the three-to-five-sigma failure of per-step replanning during contact on `tool_hang`.